# GENERATIVE AI COURSE NOTES

Number-wise notes prepared from all uploaded PDFs

**Included PDF topics:** 1.1 Introduction, 1.2 What You Will Learn, 1.3 Environment Setup, 2.1 AI vs ML vs DL vs Generative AI, 2.2 Large Language Models (LLMs), and 2.3 Transformer Architecture.

*Note: Some PDFs were uploaded twice. Duplicate copies are included only once in these notes. The notes follow the topics and explanations shown in the supplied slides; brief clarifying wording is added to make revision easier.*

## CONTENTS


1.1 — Introduction

1.2 — What You Will Learn (Course Roadmap)

1.3 — Environment Setup: Python, IDEs & Development Tools

2.1 — AI vs ML vs DL vs Generative AI

2.2 — Large Language Models (LLMs)

2.3 — Transformer Architecture

Quick Revision — Important Differences and Workflow

# 1.1 — Introduction

This section introduces the purpose of the course and the kind of practical work learners will do.

### Why this course?

• **Hands-on:** Focuses on building real projects instead of learning only theory.

• **Career-focused:** Covers skills intended to be relevant to industry work.

• **Future-ready:** Introduces modern and emerging AI tools.

### What you will build

• **Chatbot:** A conversational AI application that interacts with users.

• **RAG:** A retrieval-based system that can use relevant information to help generate an answer.

• **Capstone project:** A larger, practical project that brings together course concepts.

• **AI agents:** Systems designed to carry out tasks more autonomously.

• **Deployment:** Making an application available to run in a production-like environment; the slides mention Docker, AWS and FastAPI.

• **MCP:** Model Context Protocol, listed in the course as a topic for connecting models with tools or context.

# 1.2 — What You Will Learn (Course Roadmap)

The course roadmap is divided into ten stages. The sequence moves from basic concepts to implementation and projects.

| No. | Topic | Main focus |
|---|---|---|
| 1 | Introduction | Course overview and orientation |
| 2 | GenAI Foundations | Core concepts and theory |
| 3 | LLMs in Python | Hands-on coding with language models |
| 4 | Prompt Engineering | Learning how to write effective prompts |
| 5 | GenAI Chatbots | Building conversational AI |
| 6 | RAG | Retrieval-Augmented Generation |
| 7 | AI Agents | Autonomous systems |
| 8 | LLM Deployment | Running models in production environments |
| 9 | MCP | Model Context Protocol |
| 10 | Capstone Projects | Real-world applications |

**Remember:** The roadmap starts with foundations, then moves to coding, prompting, building applications, deployment, MCP and a final project.

# 1.3 — Environment Setup: Python, IDEs & Development Tools

This PDF lists the main tools used during the course.

• **Python:** The programming language used for course coding and AI/LLM tasks.

• **JupyterLab:** An interactive environment for writing and running code in notebook cells.

• **Google Colab:** A browser-based notebook environment for running Python notebooks.

• **PyCharm:** An IDE used to write, organize, run and debug Python projects.

• **Visual Studio Code (VS Code):** A code editor that supports Python and many development extensions.

• **Docker:** A tool for packaging applications and their dependencies into containers.

• **AWS EC2:** A cloud virtual-machine service that can be used to run applications.

**Basic setup flow:** Install/use Python → choose an IDE or notebook → write and run code → use Docker or a cloud machine when deployment is required.

# 2.1 — AI vs ML vs DL vs Generative AI

These terms are related, but they describe different levels or capabilities of artificial intelligence.

| Term | Meaning in the slides | Examples shown |
|---|---|---|
| Artificial Intelligence (AI) | The science of making machines act smart, like humans. | Siri, Alexa, Chess Bot |
| Machine Learning (ML) | AI that learns patterns from data to make predictions. | Email spam filter, Netflix recommendations, credit-card fraud detection |
| Deep Learning (DL) | ML that uses multi-layered neural networks for complex tasks. | Face ID unlock, speech-to-text, image classification |
| Generative AI (GenAI) | AI that creates new content such as text, images and music. | ChatGPT, GitHub Copilot, DALL·E |

• **Relationship:** AI is the broad field. ML is a way for AI systems to learn from data. DL is a type of ML based on multi-layer neural networks. GenAI focuses on creating new content.

• **Easy memory trick:** AI = smart behaviour; ML = learn patterns; DL = learn with deep neural networks; GenAI = generate content.

# 2.2 — Large Language Models (LLMs)

## Meaning

A Large Language Model (LLM) is a type of AI model trained to understand and generate human-like text.

## Popular LLMs and applications listed in the slides

• **GPT-5:** Chatbots

• **Gemini 2.5:** Document question answering (Doc QA)

• **Llama 4:** Coding

• **DeepSeek:** Agents

## What makes an LLM “large”?

• **Parameters:** The numerical values a model learns during training. They help determine how the model represents patterns.

• **Scale:** The slides give GPT-3 as an example with 175 billion (175B) parameters.

• **Trade-off:** More parameters can support better understanding, but require significantly more computational resources.

• The slides state that newer models such as GPT-4 and GPT-5 likely exceed GPT-3's parameter count.

### LLM workflow — step by step

| Step | Stage | What happens |
|---|---|---|
| 1 | Input text | Raw text enters the system. |
| 2 | Tokenization | Text is converted into numerical tokens. |
| 3 | Neural network | The Transformer processes relationships between tokens. |
| 4 | Output tokens | The model predicts a sequence of next tokens. |
| 5 | Output text | Tokens are converted back into readable text. |

### Tokenization

• Tokenization breaks text into smaller units: words, subwords or characters.

• These units are converted into numerical IDs so the model can process text.

• It can help handle unfamiliar words by splitting them into smaller meaningful parts and supports a reduced vocabulary size.

• Slide example: “ChatGPT is amazing!” may be split into [“Chat”, “G”, “PT”, “ is”, “ amazing”, “!”].

• Actual splits vary depending on the model and tokenizer (for example, GPT-2, GPT-4 or LLaMA).

### Proprietary vs Open-Source LLMs (as categorized in the slides)

• **Proprietary examples:** OpenAI GPT models, Google Gemini 2.5, Anthropic Claude Opus 4.1 and Claude Sonnet 4.

• **Open-source examples:** Meta Llama 2 and Llama 3, Google Gemma 2 and Gemma 3, OpenAI GPT-oss, and DeepSeek.

*Study note: These are examples and labels as presented in the uploaded course material; model versions and licensing can change over time.*

## 2.3 — Transformer Architecture

### What is a Transformer?

• Transformers process an entire sentence at once rather than reading only one word at a time.

• They use **self-attention** to capture the importance of words and the context between them.

• The architecture was introduced in the 2017 paper “Attention Is All You Need”.

• Transformers are useful for tasks such as translation and text generation.

### Core architecture: Encoder and Decoder

| Encoder | Decoder |
|---|---|
| Converts input text into rich contextual embeddings through stacked layers. | Generates output sequences step by step, one token at a time. |
| Each layer combines self-attention and feed-forward networks. | Uses masked self-attention to prevent access to future tokens. |
| Supports bidirectional processing to capture the full input context. | Uses cross-attention to integrate encoder information. |

### Self-Attention Mechanism

• **Purpose:** It lets the model weigh how important each word is in relation to the other words in the sequence.

• **Query (Q), Key (K), Value (V):** Vectors created from input embeddings through linear projections. They help the model represent relationships between tokens.

• **Attention computation:** The model calculates similarity scores between Query and Key, normalizes the scores with softmax, and applies them to the Values. This produces context-aware representations.

### Positional Encoding

Because a Transformer processes words together, it needs information about word order. Positional encoding adds a position signal to word embeddings. The slides describe sine and cosine patterns as a way to represent positions, allowing the model to use both word meaning and word order.

### Training Process

• **Pre-training:** The Transformer is first trained on large amounts of text.

• **Self-supervised learning:** The slides mention learning tasks such as filling in missing words (masked language modeling) or predicting the next sentence.

• **Fine-tuning:** The pre-trained model is then adjusted for specific tasks.

• By processing very large amounts of text, the model learns patterns related to grammar, meaning and language.

## Quick Revision — Important Points

• **AI → ML → DL:** AI is the broad field; ML learns patterns from data; DL uses multi-layer neural networks.

• **GenAI:** Creates new content such as text, images and music.

• **LLM:** An AI model trained to understand and generate human-like text.

• **Token:** A word, subword or character unit used by an LLM after tokenization.

• **LLM flow:** Input text → Tokenization → Transformer/neural network → Output tokens → Output text.

• **Self-attention:** Helps a model understand relationships and importance among tokens.

• **Q, K, V:** Query, Key and Value vectors used in attention.

• **Positional encoding:** Adds word-order information.

• **Encoder:** Builds contextual representations of input.

• **Decoder:** Generates output tokens step by step.

• **Pre-training:** Initial training on large text collections; **fine-tuning:** adapting a model to a particular task.

*Prepared from the six unique uploaded course PDFs. Duplicate copies of topics 2.1, 2.2 and 2.3 were not repeated.*